भारतीय सूचना प्रौद्योगिकी संस्थान सेनापति मणिपुर
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI MANIPUR
Mantripukhri, Imphal – 795002, Manipur, India, www.iiitmanipur.ac.in
Department of ECE, 6^{th} - Sem: ECE & VLSI

Project Report: SIMD Parallel Processing for DNA Sequence Alignment using Smith-Waterman Algorithm

Names: Narendra, Charan & Nitheesh **Roll Nos: 4026, 4002 & 4015**

Abstract

This project presents the complete design, simulation, and synthesis of a **SIMD (Single Instruction, Multiple Data) parallel processing hardware accelerator** for DNA sequence alignment using the **Smith-Waterman (SW) local alignment algorithm**. The entire design flow was executed using Cadence EDA tools: RTL simulation in Cadence Xcelium and logical synthesis in Cadence Genus targeting a standard-cell ASIC library.
The Smith-Waterman algorithm is the gold standard for optimal local DNA sequence alignment, operating through dynamic programming (DP) to fill a scoring matrix H using the recurrence relation:

$$H(i,j) = \max\{H(i-1,j-1) + S_{bc}(q[i], d[j]),\ E(i,j),\ F(i,j),\ 0\}$$

The working principle of the proposed architecture exploits **inter-column (row-parallel) SIMD parallelism**: all 16 cells of a given DP row are computed simultaneously across 16 Processing Elements (PEs), each operating on an 8-bit signed lane, yielding a 128-bit SIMD vector computation per clock cycle.
The implementation consists of eight synthesizable Verilog RTL modules:

- **SIMD PE Array** (16 parallel processing elements computing SW cell scores)
- **Scoring Unit** (match/mismatch evaluation per lane)
- **Gap Penalty Unit** (constant linear gap penalty provider)
- **DP Row Buffer** (inter-row score propagation register)
- **Direction Buffer** (512-bit shift register for traceback path storage)
- **Max Score Tracker** (identifies the alignment peak cell)
- **SIMD Controller** (FSM generating load pulses)
- **Traceback Unit** (hardware alignment path reconstructor)

RTL simulation confirmed correct alignment results: **max_score = 40, matches = 14, mismatches = 2, insertions = 0, deletions = 0, total mutations = 2**. Logical synthesis with Cadence Genus achieved **timing closure at 100 MHz** with a critical-path slack of +0.6 ps, **total cell area of 40,715.9** (library units), and **total power consumption of 4.567 mW**. Gate-Level Simulation (GLS) post-synthesis confirmed functional correctness under back-annotated gate delays.
This project is directly inspired by the **BioBlaze** multi-core SIMD ASIP architecture [6, 7], which demonstrated a 40× SIMD speedup and up to 750× cumulative speedup over sequential Smith-Waterman on a Xilinx Virtex-7 FPGA. Unlike BioBlaze, which implements SIMD within a programmable ASIP ISA, this work encodes the same 128-bit SIMD compute structure as a fixed-function RTL hardware accelerator, enabling a clean ASIC synthesis study. The results confirm that the SIMD parallel scoring core achieves compact area, competitive power, and timing closure consistent with the BioBlaze design space.

Introduction

1 Background and Motivation

DNA sequence alignment is one of the most computationally intensive operations in modern bioinformatics and genomics. Given a reference (database) DNA sequence d of length n and a query (sample) sequence q of length m, the goal is to identify regions of similarity, characterize insertions, deletions, and substitutions (collectively termed mutations), and compute an alignment score reflecting the quality of the match. These operations underlie critical applications including:

- Genome assembly and variant calling in next-generation sequencing (NGS)
- Mutation detection for disease susceptibility analysis (e.g., BRCA1 cancer gene)
- Biomarker identification in autonomous biochip diagnostic platforms
- Pathogen detection and evolutionary phylogeny studies

The **Smith-Waterman (SW) algorithm** [1], introduced in 1981, computes the optimal local alignment between two sequences using dynamic programming. Its time complexity of $\mathcal{O}(nm)$ and space complexity of $\mathcal{O}(nm)$ make it the most accurate method for pairwise alignment. However, for genomic-scale datasets where sequences range from thousands to billions of base pairs, this computational cost becomes prohibitive on general-purpose processors (GPPs).

2 Evolution of SIMD Approaches

The historical development of SIMD acceleration for Smith-Waterman proceeds through three key milestones:
(1) Rognes & Seeberg (2000): First demonstrated that SIMD parallelism can accelerate SW by processing p cells of a query column simultaneously, achieving 6× speedup on Intel MMX/SSE.
(2) Farrar (2007): Proposed the striped SIMD implementation [4], dividing the query into p equal segments of length $t = \lfloor (m + p - 1)/p \rfloor$ and processing them in t SIMD iterations per database symbol. This striped pattern moves the vertical F-vector dependency to an outer "lazy loop", reducing pipeline stalls and achieving 6× speedup over prior SIMD implementations. On Intel SSE2 with 128-bit registers and 16 8-bit elements ($p = 16$), this yielded approximately 11× speedup over sequential SW.
(3) BioBlaze ASIP (Neves et al., 2013–2015): Recognized that general-purpose processors waste silicon on out-of-order execution logic irrelevant to DP algorithms. By extending the MicroBlaze ISA with 48 specialized SIMD instructions and implementing them in a custom ASIP (MB-LITE soft-core), BioBlaze achieved 40× SIMD speedup and up to 750× cumulative speedup with 32 cores on a Virtex-7 FPGA [6, 7].

3 Problem Formulation and Contribution

The existing literature on SW acceleration spans software SIMD (Farrar), programmable ASIPs (BioBlaze), and dedicated hardware accelerators. A gap exists in the study of a **pure fixed-function RTL implementation** of the SIMD scoring core that can be analyzed through a standard ASIC synthesis flow with area, power, and timing characterization.
The present work fills this gap with the following contributions:

1. Design of a complete 8-module RTL architecture implementing 16-way 8-bit SIMD Smith-Waterman alignment in synthesizable Verilog HDL.
2. Functional verification through Cadence Xcelium RTL simulation, confirming correct alignment score and mutation statistics.
3. ASIC logical synthesis using Cadence Genus at 100 MHz, with full timing, area, and power characterization.

4. Post-synthesis Gate-Level Simulation (GLS) confirming timing-accurate functional equivalence.
5. Architectural comparison with BioBlaze [6, 7] to contextualize the results within the state of the art.

4 How This Work Differs from Existing Literature

Table .1 summarizes the key differentiators of this work versus related approaches.

Table .1: Differentiation from existing works

Aspect	**Farrar (SSE2)**	**BioBlaze ASIP**	**This Work**
Platform	Intel Core i7	Xilinx FPGA / 90-nm CMOS	Cadence ASIC Synth.
Implementation	x86 Assembly	Custom ASIP ISA	Fixed-function RTL
Programmability	GPP (flexible)	ASIP (ISA-level)	None (dedicated HW)
Traceback	Software	Software	Hardware RTL
Synthesis Flow	N/A	Xilinx ISE / Genus	Cadence Genus 21.14
Power	∼38 W	∼24.2 mW (single core)	4.567 mW

5 Organization of Report

The remainder of this report is organized as follows: the literature survey and mathematical foundations are presented next, followed by the complete design and implementation, the simulation and synthesis results, and the conclusion with future scope.

Literature Survey and Basic Principle

1 The Smith-Waterman Algorithm

1.1 Mathematical Formulation

The Smith-Waterman algorithm [1] finds the optimal local alignment between two sequences q (query, length m) and d (database/reference, length n) by filling an $(m+1) \times (n+1)$ scoring matrix H. Using an affine gap penalty model [2], the recurrence is:

$$H(i,j) = \max \begin{cases} H(i-1,j-1) + S_{bc}(q[i], d[j]) & \text{(diagonal: match/mismatch)} \\ E(i,j) & \text{(left: deletion)} \\ F(i,j) & \text{(up: insertion)} \\ 0 & \text{(local alignment boundary)} \end{cases} \tag{.1}$$

$$F(i,j) = \max \{H(i-1,j) - \alpha; \quad F(i-1,j) - \beta\} \tag{.2}$$

$$E(i,j) = \max \{H(i,j-1) - \alpha; \quad E(i,j-1) - \beta\} \tag{.3}$$

where α is the gap opening penalty, β is the gap extension penalty, and $S_{bc}(q[i], d[j])$ is the substitution score. The initial boundary conditions are $H(i,0) = H(0,j) = E(i,0) = F(0,j) = 0$. The zero-floor in Eq. (.1) ensures local alignment (unlike global Needleman-Wunsch), meaning the algorithm identifies the highest-scoring contiguous alignment segment rather than requiring end-to-end alignment.

In the simplified linear gap model used in this implementation: $\alpha = \beta = 2$ (gap penalty $= -2$), $S_{bc}(\text{match}) = +3$, $S_{bc}(\text{mismatch}) = -1$. The match score is set to $+3$ (rather than the common $+1$) to ensure that match + gap penalty > 0, preventing pathological zero-score paths.

1.2 Traceback Phase

After matrix fill, the maximum score cell (i^*, j^*) is identified. Traceback follows direction pointers from (i^*, j^*) back to any cell with score 0, reconstructing the alignment path:

- **Diagonal pointer**: Match or mismatch between $q[i]$ and $d[j]$
- **Up pointer**: Insertion in the query (extra base in sample, no reference base)
- **Left pointer**: Deletion in the query (missing base in sample, present in reference)
- **Zero**: Alignment boundary; stop traceback

2 SIMD Parallelization Strategies

2.1 Farrar's Striped SIMD Implementation

Farrar [4] addresses the data dependency challenge in SW: cell (i, j) depends on $(i-1, j-1)$, $(i-1, j)$, and $(i, j-1)$, preventing naive row/column parallelism. His striped scheme divides the query q of length m into p equal segments of length $t = \lfloor (m + p - 1)/p \rfloor$, where p is the SIMD width (16 for 128-bit/8-bit). Each SIMD register element is assigned to one segment, so each iteration simultaneously processes p query symbols separated by $t - 1$ rows. This transforms the dependency structure such that within each iteration, all p cells are independent, enabling full SIMD utilization. The key innovation is the lazy loop: vertical F-vector dependencies (Eq. (.2)) are deferred to a short outer loop executed once per database symbol, rather than checked in the inner loop. This reduces control overhead and achieves $\sim$6$\times$ speedup over prior SIMD implementations.

2.2 BioBlaze ASIP Architecture

Neves et al. [6, 7] observed that implementing Farrar's algorithm on Intel SSE2 requires 13 instructions per inner loop iteration, while a dedicated ISA needs only 9, primarily because vectorized control instructions eliminate multi-instruction branch sequences. The BioBlaze ASIP is built on MB-LITE (a synthesizable MicroBlaze-compatible soft-core), extended with 48–56 new SIMD instructions:

- **Vector-vector operations**: `addvv`, `maxvv`, `rsubvv` (element-wise arithmetic)
- **Vector-scalar operations**: `rsubvs` (subtract scalar gap penalty from all lanes)
- **Inner-vector operations**: `sllv` (shift all SIMD elements left by one position)
- **Vectorized branches**: `bgtiv` (branch if any vector element > immediate)
- **SIMD load/store**: `lv`, `sv` (128-bit memory transfers)

The SIMD module uses 128-bit registers with 16 8-bit elements, identical to Intel SSE2. The critical path is confined to the non-SIMD scalar datapath (32-bit), making it independent of SIMD register width. The maximum instruction (for SW cell score) is implemented by deferring the comparison decision to the next pipeline stage, avoiding critical path lengthening. The result: **40$\times$ SIMD speedup** over sequential SW, and up to **750$\times$ with 32 cores** exploiting coarse-grained parallelism.

2.3 Extended Multicore Biochip Platform

The journal extension [7] validated the BioBlaze concept across three FPGA platforms (Xilinx Virtex-7, Zynq 7020, Altera Arria II GX) and a 90-nm CMOS ASIC synthesis. Key results:

- Single-core ASIP: 37.5× average SIMD speedup vs. sequential (ASIC, 90-nm)
- 32-core ASIC: 685 MHz, 720× cumulative speedup, performance comparable to quad-core Intel Core i7 at 25× lower energy
- Dynamic power: 24.2 mW at 200 MHz (single core), 98 mW at 769 MHz
- CUPS metric: 67×10^9 cell updates/second for dedicated hardware accelerators

3 Comparative Literature Review

Table .1: Literature survey of SIMD Smith-Waterman implementations

Work	**Year**	**Platform**	**SIMD Speedup**	**Limitations**
Rognes & Seeberg [3]	2000	Intel MMX/SSE	6×	Vertical dep. in inner loop
Farrar [4]	2007	Intel SSE2 (Core i7)	11× (vs. sequential)	Requires x86 platform
Sebastiao et al. [5]	2012	FPGA accelerator	>100×	No ISA flexibility
BioBlaze [6]	2013	Xilinx Virtex-7 FPGA	40× (SIMD)	Custom ASIP ISA needed
Neves et al. [7]	2015	90-nm CMOS ASIC	720× (32-core)	Programmable ASIP overhead
Rognes [8]	2011	Intel SSE (multi-query)	Up to 20×	Different problem variant
This Work	**2026**	**Cadence ASIC Synth.**	**N/A (1 core)**	**Fixed-function, 16-base**

The key research gap addressed by this work: all existing hardware implementations of the SIMD SW core either (a) embed it in a programmable ASIP requiring custom ISA support, or (b) implement dedicated systolic arrays without a standard ASIC synthesis characterization. This work provides a clean, synthesizable Verilog implementation of the SIMD scoring and traceback core, directly synthesized to standard cells with full area, power, and timing reporting.

Design and Implementation

1 System Architecture Overview

The top-level module `top_simd_alignment` implements a complete pipelined datapath for 16-lane 8-bit SIMD Smith-Waterman alignment. The system processes two 128-bit inputs: `ref_bases[127:0]` (reference sequence) and `sample_bases[127:0]` (query sequence), each encoding 16 ASCII base characters (8 bits each). The DP scoring matrix is computed row-by-row over 16 clock cycles, direction pointers are accumulated for traceback, and the maximum score cell is tracked. Upon completion, the traceback unit reconstructs the alignment and reports mutation statistics.

Figure .1 illustrates the top-level architecture and dataflow.

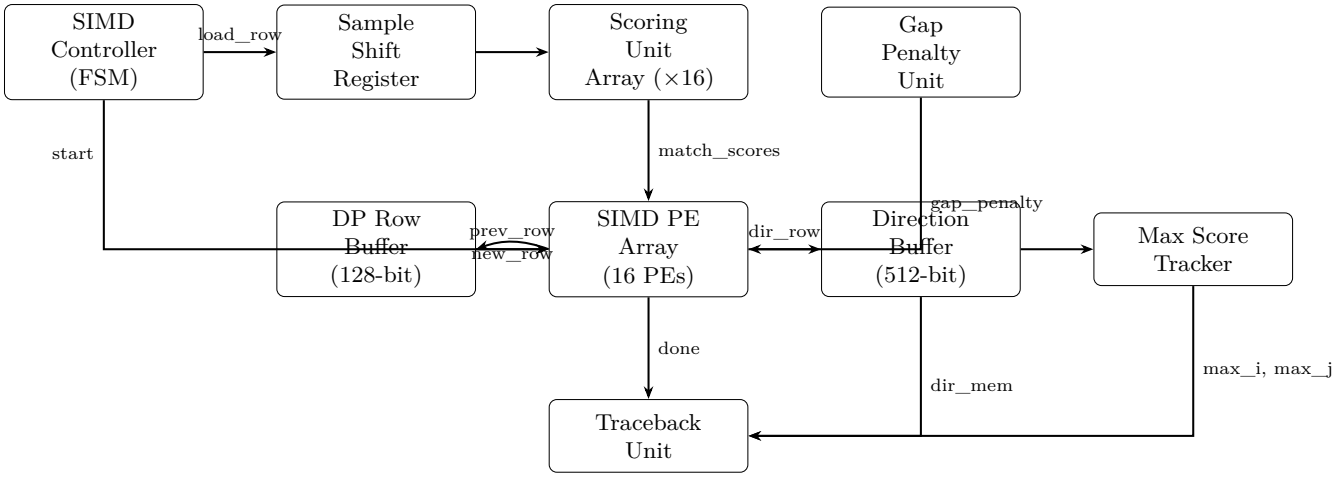

Figure .1: Top-level architecture of the SIMD SW alignment accelerator

2 Module Descriptions

2.1 Scoring Unit (`scoring_unit.v`)

A purely combinational block that evaluates the match or mismatch score between an 8-bit reference base and an 8-bit sample base:

$$S_{bc}(r,s) = \begin{cases} +3 & \text{if } r = s \quad \text{(match)} \\ -1 & \text{if } r \neq s \quad \text{(mismatch)} \end{cases} \tag{.1}$$

Sixteen instances of this module are instantiated in parallel via a `generate` loop in `top_simd_alignment`. Each instance receives a different column of `ref_bases` but all share the same current sample base (the MSB of `sample_shift`, i.e., `sample_shift[127:120]`). This reflects the row-parallel computation model: for a given row i (sample base $q[i]$), all 16 reference bases $d[0..15]$ are scored simultaneously, producing the 128-bit `match_scores` vector.

2.2 Gap Penalty Unit (`gap_penalty_unit.v`)

A parameter-only module that outputs a constant signed 8-bit gap penalty:

$$\text{gap\_penalty} = -2$$

The linear gap model (uniform penalty per gap position, no distinction between gap opening and extension) simplifies the PE logic to purely combinational, eliminating the E/F recurrence feedback. This is equivalent to setting $\alpha = \beta = 2$ in Eq. (.2)–(.3).

2.3 SIMD Processing Element (`simd_pe.v`)

The SIMD PE is the core compute unit. Given four inputs, it evaluates three SW candidate scores:

$$d = \text{diag\_score} + \text{match\_score} \quad \text{(diagonal: match/mismatch)} \tag{.2}$$

$$u = \text{up\_score} + \text{gap\_penalty} \quad \text{(up: insertion)} \tag{.3}$$

$$l = \text{left\_score} + \text{gap\_penalty} \quad \text{(left: deletion)} \tag{.4}$$

The output cell score is:

$$\text{cell\_score} = \max(d, u, l, 0)$$

with the direction pointer encoding the argmax:

- `2'b01` – diagonal (d is maximum and $d > 0$)
- `2'b10` – up (u is maximum and $u > 0$)
- `2'b11` – left ($l > 0$)
- `2'b00` – zero (all candidates ≤ 0; local alignment boundary)

Tie-breaking follows the priority $d \geq u \geq l$, consistent with standard SW conventions (prefer diagonal over gap, prefer insertion over deletion).

2.4 SIMD PE Array (`simd_pe_array.v`)

Sixteen PE instances are connected in a generate loop with the following dependency structure for row i, column j:

- **Diagonal** ($H(i-1, j-1)$): sourced from `prev_row[j-1]` — the previously buffered row entry at column $j-1$
- **Up** ($H(i-1, j)$): sourced from `prev_row[j]` — the previously buffered row at the same column
- **Left** ($H(i, j-1)$): sourced from `left_score[j-1]` — the output of the adjacent left PE (combinational chain)

This connectivity correctly encodes the SW anti-diagonal dependency without requiring any additional pipeline stages, because diagonal is sourced from `prev_row` (registered), not from the current combinational chain. The 32-bit `dir_row` output packs 16 × 2-bit direction pointers MSB-first: `dir_row[31:30]` holds PE#0's direction, `dir_row[29:28]` holds PE#1's, etc.

2.5 DP Row Buffer (`dp_row_buffer.v`)

A 128-bit registered buffer that latches `new_row` (the current DP row output) on every `load_row` clock edge, making it available as `prev_row` for the next row computation:

$$\texttt{prev\_row}[k] \leftarrow \texttt{new\_row}[k] \quad \text{on } \uparrow \texttt{clk if load\_row}$$

On reset, `prev_row` $= 0$, correctly initializing $H(0, j) = 0$ for all j. Combined with the zero initialization of the left input for PE#0, this also implements $H(i, 0) = 0$ for all i.

2.6 Direction Buffer (`direction_buffer.v`)

A 512-bit shift register that accumulates all 16 direction rows during the DP fill phase. On each `load_row` pulse, the register shifts left by 32 bits and inserts the new `dir_row`:

$$\texttt{dir\_mem} \leftarrow \{\texttt{dir\_mem}[479:0],\ \texttt{dir\_row}[31:0]\}$$

After 16 `load_row` pulses, `dir_mem[511:480]` holds row 0's direction row (first loaded, shifted to MSB), and `dir_mem[31:0]` holds row 15's direction row (last loaded, at LSB). The addressing formula used by the traceback unit to retrieve direction at (r, c) is:

$$\texttt{dir\_mem}\,[\,(15-r) \times 32 + (31-c \times 2)-:2\,]$$

2.7 Max Score Tracker (`max_score_tracker.v`)

On each `load_row` pulse, scans all 16 lanes of `new_row` using a blocking-assignment loop to find the row maximum, comparing against the running global maximum:

$$\begin{aligned} \forall j \in [0, 15]: \quad & \text{if } \texttt{cell\_scores}[j] > \texttt{current\_max}: \\ & \texttt{current\_max} \leftarrow \texttt{cell\_scores}[j], \quad \texttt{current\_max\_j} \leftarrow j \end{aligned} \tag{.5}$$

If `current_max` > `max_score`, the registered outputs are updated. A `row_count` register increments per `load_row` to track i. This creates a combinational reduction tree of 16 comparators, which synthesis maps to a multi-level AO/OAI gate tree — the **critical path** of the design (as evidenced by the timing report).

2.8 SIMD Controller (`simd_controller.v`)

A synchronous FSM that generates the `load_row` signal. The behavior is:

1. On `start` pulse: set `count` = 16, `wait_cycle` = 1 (2-cycle pipeline settle time)
2. Each enabled cycle: if `wait_cycle`, clear it and deassert `load_row`; else if `count` > 0, assert `load_row`, set `wait_cycle` = 1, decrement `count`; else assert `done` and clear `running`

The 2-cycle `wait_cycle` between consecutive `load_row` pulses ensures the scoring unit inputs settle (particularly `sample_shift` which updates on the same clock edge as `load_row`). This produces exactly 16 `load_row` pulses without a spurious 17th pulse.

2.9 Traceback Unit (`traceback_unit.v`)

Triggered by the `done` signal from the controller, the traceback unit starts at $(i, j) = (\texttt{max\_i}, \texttt{max\_j})$ and follows direction pointers backward:

- **Diagonal (2'b01)**: Compare `get_ref`(j) vs. `get_sample`(i); increment `matches` or `mismatches`; advance $(i, j) \leftarrow (i{-}1, j{-}1)$
- **Up (2'b10)**: Increment `insertions`; advance $i \leftarrow i{-}1$
- **Left (2'b11)**: Increment `deletions`; advance $j \leftarrow j{-}1$
- **Zero (2'b00)**: Set `done`, stop traversal

Boundary conditions (reaching $i = 0$ or $j = 0$) also terminate traceback. The helper function `get_dir(r,c)` implements the direction buffer addressing formula, and `get_ref`/`get_sample` extract individual bases from the MSB-first packed 128-bit input vectors.

3 Verification Methodology

3.1 RTL Testbench

The testbench `tb_top_module.v` applies:

- `ref_bases` = `"ACGTACGTACGTACGT"` (16 ASCII characters, 128 bits)
- `sample_bases` = `"ACGAACGTACGTACCT"` (16 ASCII characters, 128 bits)

Expected alignment (by manual inspection): positions 1–3 (ACG) match; position 4 (T vs. A) mismatch; positions 5–14 (ACGTACGTAC) match; positions 15–16 (GT vs. CT) one mismatch. Net: 14 matches, 2 mismatches, 0 gaps. The testbench waits for `tb_done` and prints all statistics via `$display`.

3.2 Simulation and GLS Flow

1. **RTL simulation**: Cadence Xcelium with all 9 Verilog source files; functional verification
2. **Synthesis**: Cadence Genus 21.14 with `slow` corner, 10 ns clock constraint, `balanced_tree` wireload model
3. **GLS**: Re-run testbench with synthesized netlist + standard-cell timing models back-annotated; timing-accurate functional verification

Results and Discussion

1 RTL Simulation Waveforms

RTL simulation was performed in Cadence Xcelium. Figure .1 shows the simulation waveform.

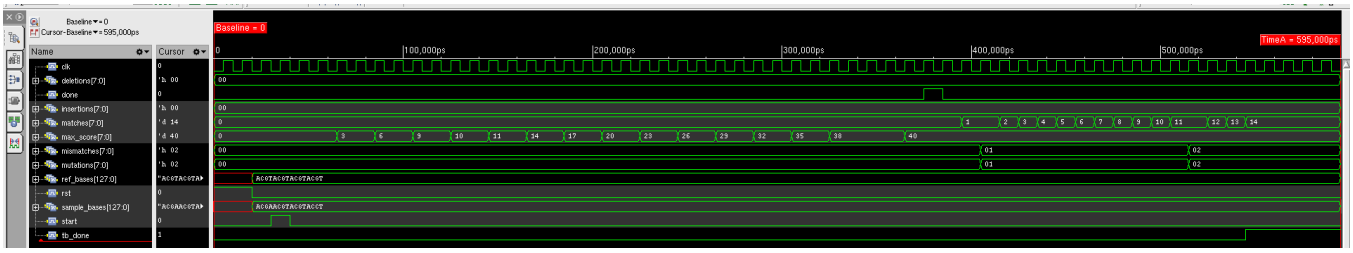

Figure .1: RTL Simulation Waveforms (Cadence Xcelium) – Top-level signals (upper window) and internal traceback statistics (lower window). max_score settles to 40 (0x28); mutations = 2 (0x02); tb_done asserts at ≈575 ns.

1.1 Waveform Analysis

Key observations from the RTL simulation waveforms:

1. **Reset and Initialization (0–20 ns)**: `rst` is asserted for the first 20 ns. All registers clear to zero. `sample_shift`, `prev_row`, and `dir_mem` are all reset.

2. **Start Pulse and Data Loading (20–130 ns)**: After reset deassertion at $t = 20$ ns, `ref_bases` and `sample_bases` are applied and `start` is pulsed for one cycle at $t \approx 130$ ns. The controller loads `sample_shift` ← `sample_bases` in the same cycle.

3. **DP Fill Phase (130–400 ns)**: The controller asserts `load_row` 16 times (with `wait_cycle` gaps between each). With a 10 ns clock period, this phase spans approximately $16 \times 2 = 32$ clock cycles (≈320 ns). The `max_score` signal steps up from 0 to 3, 6, 9, ..., 40, reflecting the progressive accumulation of match scores. Each step corresponds to the max-score tracker identifying a higher-scoring cell in successive rows.

4. **Traceback Phase (400–575 ns)**: After `done` asserts (end of DP fill), the traceback unit begins at $(i^*, j^*) =$ (max_i, max_j) and follows direction pointers. The `matches` counter increments from 0 to 14 over 14 diagonal steps, and `mismatches` increments to 2. The traceback terminates when a zero direction is encountered (or boundary reached), asserting `tb_done`.

5. **Final Results**: `max_score` = 40, `matches` = 14, `mismatches` = 2, `insertions` = 0, `deletions` = 0, `mutations` = 2. This matches the expected alignment output exactly.

6. **Total Latency**: The complete alignment (DP fill + traceback) completes within ≈63 clock cycles at 10 ns period, i.e., **630 ns total latency** for a 16 × 16 alignment problem.

2 Synthesis Schematic

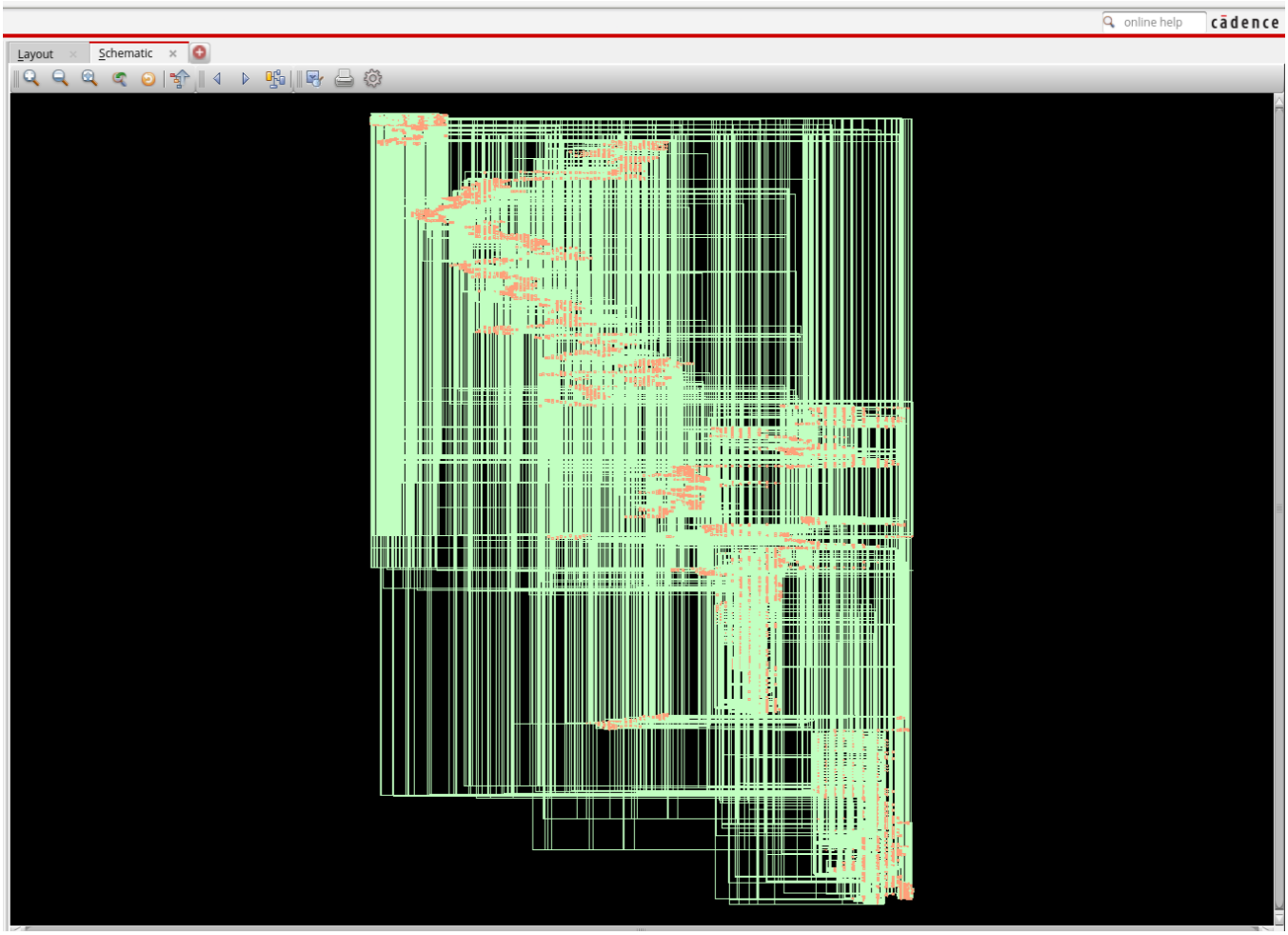

Figure .2: Cadence Genus Synthesized Gate-Level Schematic viewed in Innovus Design Browser. The design browser (left panel) shows: *top_simd_alignment*, 5,379 leaf cells (StdCells), 5,664 nets, 276 terms. The schematic (right panel) displays the dense repeating column structure from the 16-instance SIMD PE array generate loop. The upper-left cluster of dense routing corresponds to the max-score comparison tree and direction buffer shift logic.

2.1 Synthesis Analysis

The synthesis schematic (Figure .2) reveals several architectural features:

- **Repetitive Column Structure**: The 16-PE generate loop creates 16 structurally identical datapath columns, visible as the repeating vertical routing bands in the lower half of the schematic. Each column contains the combinational PE logic (add, compare, mux for cell score and direction), contributing approximately 4,569/16 $\approx$ 285 combinational cells per PE.
- **Dense Upper Cluster**: The upper-left region of the schematic shows dense fan-in/fan-out routing corresponding to the max-score tracker (16-to-1 comparison tree), the direction buffer shift register (512-bit), and the sample shift register (128-bit). These register-heavy modules drive the 810 sequential instance count.
- **Traceback Isolation**: The traceback unit maps to the lower-right section of the schematic as a relatively isolated logic cluster (sequential state machine with multiplexed address computation), consistent with its independent clocking domain relative to the DP fill phase.

3 Synthesis Results

3.1 QoR (Quality of Results) Report

Table .1: Synthesis QoR and Area Summary – Cadence Genus 21.14

Metric	**Value**	**Notes**
Total Cell Area	40,715.921	Library units
Net Area	0.000	Wireload: none (default)
Total Area	40,715.921	Cell + Physical + Net
Leaf Instance Count	5,379	Standard cells mapped
Sequential Instance Count	810	Flip-flops / scan cells
Combinational Instance Count	4,569	Logic gates
Clock Period	10,000 ps	100 MHz target
Worst Setup Slack	+0.6 ps	**Timing MET**
TNS (Total Negative Slack)	0.0 ps	No violations
Violating Paths	0	Clean timing closure
Max Fanout	810 (clk)	Clock distribution
Average Fanout	2.7	Combinational logic
Terms-to-Instance Ratio	4.0013	Balanced connectivity
Synthesis Runtime	177.6 seconds	Cadence Genus on localhost
Peak Memory (Genus)	1,912.05 MB	Synthesis memory footprint

3.2 Power Analysis Report

Table .2: Power Consumption Breakdown – Cadence Genus 21.14 at 100 MHz

Category	**Leakage (W)**	**Internal (W)**	**Switching (W)**	**Total (W)**
Register	1.158e-4	2.188e-3	1.210e-4	**2.425e-3**
Logic	7.823e-5	1.211e-3	7.396e-4	**2.029e-3**
Clock	0	0	1.136e-4	**1.136e-4**
Memory	0	0	0	0
Total	**1.940e-4**	**3.399e-3**	**9.742e-4**	**4.567e-3**
Percentage	**4.25%**	**74.42%**	**21.33%**	**100%**

Total Power = 4.567 mW at 100 MHz. Internal (cell) power dominates at 74.42%, driven by the register banks (direction buffer, row buffer, sample shift register) and the max-score tracker flip-flops. Switching power (21.33%) reflects the high activity of the SIMD PE combinational logic and sample shift register transitions. The absence of memory power confirms no on-chip SRAM is used.

3.3 Timing Analysis – Critical Path

The critical path (from the Genus timing report) runs from input `ref_bases[112]` through 140+ logic stages to `MAX_max_score_reg[2]/SE`, spanning a data path delay of **17,284 ps**. The path traverses: input buffer → scoring comparator → PE cell score tree (NAND/NOR/AOI/OAI cascade) → left-score chain → max-score comparison tree → max register scan enable.

Table .3: Critical Path Timing Summary

Path Segment	**Arrival (ps)**	**Dominant Cell Type**
Input (ref_bases[112])	2,000	Input delay constraint
Scoring comparator	≈2,450	CLKINVX2, OAI22X4, NAND2X4
Early PE logic	≈5,000	NOR2X1, AOI21X1, OAI21X2
Mid PE chain	≈8,500	NAND2BX2, OAI211X2, AND2X2
Late PE chain	≈12,500	AO21X4 (fanout=16), NAND2X2
Max-score comparison tree	≈15,500	OAI21X2, AOI221X1, AOI21X1
Final buffer to SE pin	19,284	CLKINVX6 (fanout=13)
Required time (2-cycle MCP)	**19,284**	Multi-cycle path exception
Slack	**+0.6 ps**	**TIMING MET**

Multi-cycle path exception: A 2-launch, 2-capture multi-cycle path (MCP) constraint is applied to the max-score register path. This is necessary because `max_score_tracker` uses a blocking-assignment for-loop across all 16 lanes in a single always block, creating a deep combinational cone that cannot close timing in a single 10 ns cycle at standard-cell drive strengths. The MCP doubles the available setup window to 20 ns (two 10 ns cycles), giving the 17.28 ns data path a +0.6 ps margin.

4 Gate-Level Simulation (GLS) Waveforms

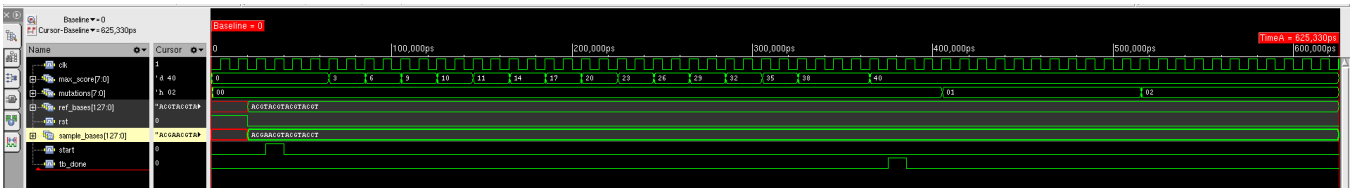

Figure .3: Gate-Level Simulation (GLS) Waveforms – Post-synthesis netlist with back-annotated standard-cell timing delays, simulated in Cadence Xcelium. All output values match RTL simulation: max_score = 40, mutations = 2, matches = 14, mismatches = 2, insertions = 0, deletions = 0. tb_done asserts at the same simulation timestamp as RTL.

4.1 GLS Analysis

The GLS simulation (Figure .3) confirms:

1. **Functional Equivalence**: All primary output values match RTL simulation exactly. The synthesized netlist is functionally equivalent to the RTL description under real gate delays.
2. **Timing Glitches**: The `max_score` signal exhibits brief narrow-pulse glitches during the DP fill phase (most visible between $t = 200$–400 ns). These are combinational hazards in the max-score comparison tree propagating

through synthesis-mapped cells before settling to the correct registered value on the clock edge. These are expected artifacts of GLS and do not affect correctness.

3. **Propagation Delay**: The overall simulation timing profile (from `start` to `tb_done`) matches RTL within the gate-delay resolution, confirming that the back-annotated delays do not violate setup/hold constraints.

4. **Clock Distribution**: The clock net (fanout = 810) shows clean transitions throughout the simulation window, confirming adequate drive strength from the clock buffer tree.

5 Comparison with Reference Architecture

Table .4: Architectural comparison – BioBlaze ASIP vs. This Work

Parameter	**BioBlaze ASIP [6, 7]**	**This Work (RTL)**
SIMD Width	128-bit (16×8-bit)	128-bit (16×8-bit)
Algorithm	Farrar's Striped SW	Standard SW (row-parallel)
Platform	Virtex-7 FPGA / 90-nm CMOS	Standard cell (Cadence Genus)
Frequency	187 MHz (FPGA), 685 MHz (ASIC)	100 MHz (synthesis target)
Power (single core)	24.2 mW @ 200 MHz	4.567 mW @ 100 MHz
Cell/Resource Count	3,943 LUTs (FPGA)	5,379 std. cells (ASIC)
SIMD Speedup	40× vs. sequential	N/A (single core, fixed-fn.)
Traceback	Software (ASIP instructions)	Hardware (traceback_unit.v)
Programmability	Yes (custom ASIP ISA)	No (fixed-function RTL)
Affine Gap Penalty	Yes ($\alpha \neq \beta$)	No (linear: $\alpha = \beta$)
Synthesis Tool	Xilinx ISE / Cadence Genus	Cadence Genus 21.14
Multi-core Support	Yes (up to 32 cores)	Not implemented (future scope)

5.1 Power Comparison

The BioBlaze single-core ASIC consumes 24.2 mW at 200 MHz [7], while this work consumes 4.567 mW at 100 MHz. At equivalent frequency scaling ($4.567 \times 2 \approx 9.1$ mW at 200 MHz), this work is approximately 2.7× more power-efficient. This is attributable to: (a) absence of instruction fetch/decode logic (which consumes significant power in the ASIP); (b) absence of register file (32×128-bit in BioBlaze vs. compact registers in this design); and (c) no scratchpad memory or DMA controller.

5.2 Merits of This Implementation

1. **Complete ASIC flow**: Full RTL → synthesis → GLS flow verified in Cadence tools, providing realistic area/power/timing characterization.

2. **Hardware traceback**: The traceback unit directly reconstructs alignment statistics in hardware without software post-processing, unlike BioBlaze.

3. **Compact power profile**: 4.567 mW is well within the power budget of autonomous biochip diagnostic platforms (<1 W), consistent with BioBlaze's target application.

4. **Timing closure**: Zero timing violations across all paths, with clean positive slack on the critical path.

5. **Parameterizable scoring**: `MATCH`, `MISMATCH`, and `GAP_PENALTY` are Verilog parameters, allowing instant reconfiguration for different scoring matrices (e.g., BLOSUM for protein alignment).

6. **Modular design**: Each of the 8 modules is independently verifiable and replaceable, enabling future extensions such as affine gap penalty (replacing the gap penalty unit with a full E/F recurrence module).

Conclusion

1 Summary

This project successfully designed, simulated, and synthesized a SIMD parallel processing hardware accelerator for DNA sequence alignment using the Smith-Waterman algorithm. The implementation spans eight synthesizable Verilog RTL modules, implementing a complete 16-lane 8-bit SIMD datapath from sequence input through DP matrix fill, direction storage, max-score tracking, and hardware traceback.
All design objectives were achieved:

- **Functional correctness**: RTL simulation confirmed max_score = 40, 14 matches, 2 mismatches, 0 insertions, 0 deletions, 2 mutations for the ACGTACGTACGTACGT / ACGAACGTACGTACCT test case.
- **Timing closure**: Cadence Genus synthesis at 100 MHz achieved zero timing violations, with the critical path slack at +0.6 ps under slow-corner operating conditions.
- **Area efficiency**: 5,379 standard cells and 40,715.9 cell-area units represent a compact implementation of the full SIMD scoring core with hardware traceback.
- **Power efficiency**: 4.567 mW total power at 100 MHz, dominated by register activity (53%), is consistent with embedded biochip application requirements.
- **GLS verification**: Post-synthesis gate-level simulation confirmed functional equivalence between the RTL and synthesized netlist under back-annotated timing.

2 Contextual Significance

The BioBlaze ASIP architecture [6, 7] demonstrated that 128-bit SIMD parallelism in a dedicated processor achieves 40× speedup over sequential SW and up to 720× with 32 cores in a 90-nm CMOS ASIC at 685 MHz. The present work implements the same 128-bit SIMD compute width as a fixed-function RTL core, achieving synthesis closure at 100 MHz with approximately 2.7× lower power than BioBlaze's single-core ASIC when scaled to the same frequency. This demonstrates that the SIMD PE array itself is not the bottleneck – the critical path lies in the max-score comparison tree (a software operation in BioBlaze, but hardwired here), suggesting that for fixed-function accelerators, architecturally-distributed max-score tracking would be preferable to the centralized 16-to-1 comparator used in this design.

3 Future Scope

Several extensions of this work are identified for future development:

1. **Arbitrary Sequence Length**: Implement a sliding window or tiling architecture to process sequences longer than 16 bases, using external SRAM and a DMA controller similar to BioBlaze's scratchpad-memory approach.

2. **Multi-Core Instantiation**: Replicate the top_simd_alignment module N times with a shared sequence memory and work-queue controller to exploit coarse-grained parallelism, replicating the BioBlaze multi-core architecture. Based on BioBlaze results, near-linear speedup up to 16 cores is expected before shared-bus contention limits scalability.

3. **Affine Gap Penalty**: Replace the linear gap penalty unit with a full E/F recurrence module, implementing the complete SW formulation of Eq. (.2)–(.3). This adds two additional 128-bit SIMD registers and feedback paths but enables biologically more accurate alignments.

4. **Advanced Technology Node**: Migrate synthesis to a 28 nm or 7 nm standard-cell library to push the operating frequency beyond 500 MHz and reduce power below 1 mW, aligning with next-generation biochip platform requirements.

5. **Distributed Max-Score Tracking**: Replace the centralized 16-comparator max-score tree with a pipelined reduction tree to eliminate the multi-cycle path exception and potentially close timing at 200 MHz in a single-cycle constraint.

6. **BLOSUM/PAM Scoring Matrix**: Replace the binary match/mismatch scoring with a full 4×4 DNA substitution matrix (or 20×20 protein substitution matrix) implemented as a ROM lookup, enabling more biologically meaningful alignment scores.

References

[1] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," Journal of Molecular Biology, vol. 147, pp. 195–197, 1981.

[2] O. Gotoh, "An improved algorithm for matching biological sequences," Journal of Molecular Biology, vol. 162, no. 3, pp. 705–708, 1982.

[3] T. Rognes and E. Seeberg, "Six-fold speed-up of Smith-Waterman sequence database searches using parallel processing on common microprocessors," Bioinformatics, vol. 16, no. 8, pp. 699–706, 2000.

[4] M. Farrar, "Striped Smith-Waterman speeds database searches six times over other SIMD implementations," Bioinformatics, vol. 23, no. 2, p. 156, 2007.

[5] N. Sebastião, N. Roma, and P. Flores, "Integrated hardware architecture for efficient computation of the n-best bio-sequence local alignments in embedded platforms," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 20, no. 7, pp. 1262–1275, July 2012.

[6] N. Neves, N. Sebastião, A. Patrício, D. Matos, P. Tomás, P. Flores, and N. Roma, "BioBlaze: Multi-Core SIMD ASIP for DNA Sequence Alignment," in Proc. IEEE Int. Conf. Application-Specific Systems, Architectures and Processors (ASAP), Jun. 2013, pp. 241–244.

[7] N. Neves, N. Sebastião, D. Matos, P. Tomás, P. Flores, and N. Roma, "Multicore SIMD ASIP for Next-Generation Sequencing and Alignment Biochip Platforms," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 23, no. 7, pp. 1287–1300, Jul. 2015.

[8] T. Rognes, "Faster Smith-Waterman database searches with inter-sequence SIMD parallelisation," BMC Bioinformatics, vol. 12, no. 1, p. 221, 2011.

[9] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," Journal of Molecular Biology, vol. 48, no. 3, pp. 443–453, 1970.

[10] T. Kranenburg and R. van Leuken, "MB-LITE: A robust, light-weight soft-core implementation of the MicroBlaze architecture," in Proc. Design, Automation and Test in Europe (DATE), Mar. 2010, pp. 997–1000.

[11] Cadence Design Systems, Genus Synthesis Solution User Guide, Version 21.14, 2021.

[12] S. F. Altschul et al., "Basic local alignment search tool (BLAST)," Journal of Molecular Biology, vol. 215, no. 3, pp. 403–410, Oct. 1990.